REGULAR EXPRESSIONS

Big Picture Background

- Pattern matching is one of the fundamental operations in many computional applications.
- Given a sequence of basic symbols find of that sequence has a certain property, e.g.
  - Has the symbol sequence“CMU” somewhere in it
  - Has the same number of a’s and b’s
  - Is a palindrome,
  - Is a valid Python program.
  - Is a valid English sentence.
- In a certain sense, all computational problems can be cast as pattern matching (or pattern transduction).

Big Picture Background

- For example, a human genome sequence consists of several billion symbols representing the bases
  - A – adenine
  - T – thymine
  - C – cytosine
  - G – guanine
- You can search for segments representing various genes (typically hundreds of bases long).
- Sometime you need to make approximate searches.

String matching

- For locating first/some/all occurences of a fixed string of symbols in a much longer string, there are many very efficient algorithms.
  - Brute force
  - Rabin-Karp algorithm
  - Knuth-Morris-Pratt algorithm
  - Boyer-Moore algorithm
  - Automata-based algorithms

Automata-based algorithms

- Automata are abstract (sometimes) restricted models of computation.
- Used to model and reason about various types of computation.
  - With restrictions of on the amount of memory or time available for the computation.
- Covered in 15-251 (GTI), 15-453 (FLAC)
- We will look at the simplest automata: finite state machines to handle pattern matching.

Finite State Concepts

- Alphabet (A): Finite set of symbols
  - A={a,b}
- String (w): concatenation of 0 or more symbols.
  - abaab,
  - ε, the empty string
    - You can't see it
    - But it can be anywhere
    - Has 0 length

Finite State Concepts

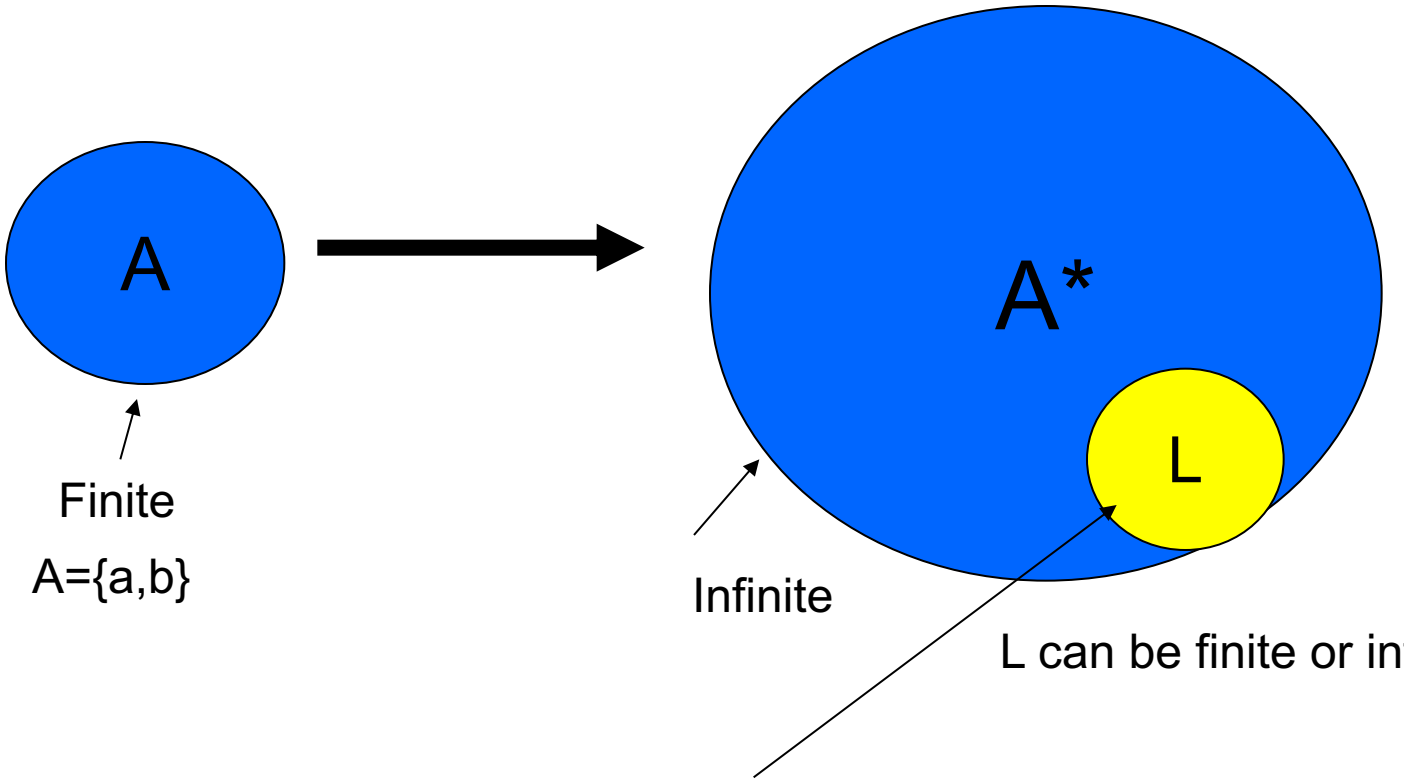
- Alphabet (A): Finite set of symbols
- String (w): concatenation of 0 or more symbols.
- A^*: The set of all possible strings
  - A*={ε,a,b,aa,ab,ba,bb,aaa,aab … }
  - (Infinite) union of all strings of length 0, 1, 2, 3, …

Finite State Concepts

- Alphabet (A): Finite set of symbols
- String (w): concatenation of 0 or more symbols.
- A^*: The set of all possible strings
- (formal) Language (L): a subset of A^*
  - The set of strings with an even number of a's
    - e.g. abbaaba but not abbaa
  - The set of strings with equal number of a's and b's
    - e.g., ababab but not bbbaa

Alphabets and Languages

A*={ε,a,b,aa,ab,ba,bb,aaa,aab … }



Describing (Formal) Languages

- L = {a, aa, aab} – description by enumeration
- L = $\{a^n b^n : n \geq 0\}$ = { ε, ab, aabb, aaabbb,….}
- L = { w: w has even number of a's}
- L = $\{w : w = w^R\}$ – All strings that are the same as their reverses, e.g., a, aba, abba
- L = {w: w = x x} – All strings that are formed by duplicating strings once, e.g., abab
- L = {w: w is a syntactically correct Java program}

Describing (Formal) Languages

- L = {w : w is a valid English word}
- L = {w: w is a valid Turkish word}
- L = {w: w is a valid English sentence}

Languages

- Languages are sets. So we can do “set” things with them
  - Union
  - Intersection
  - Complement with respect to the universe set A*.

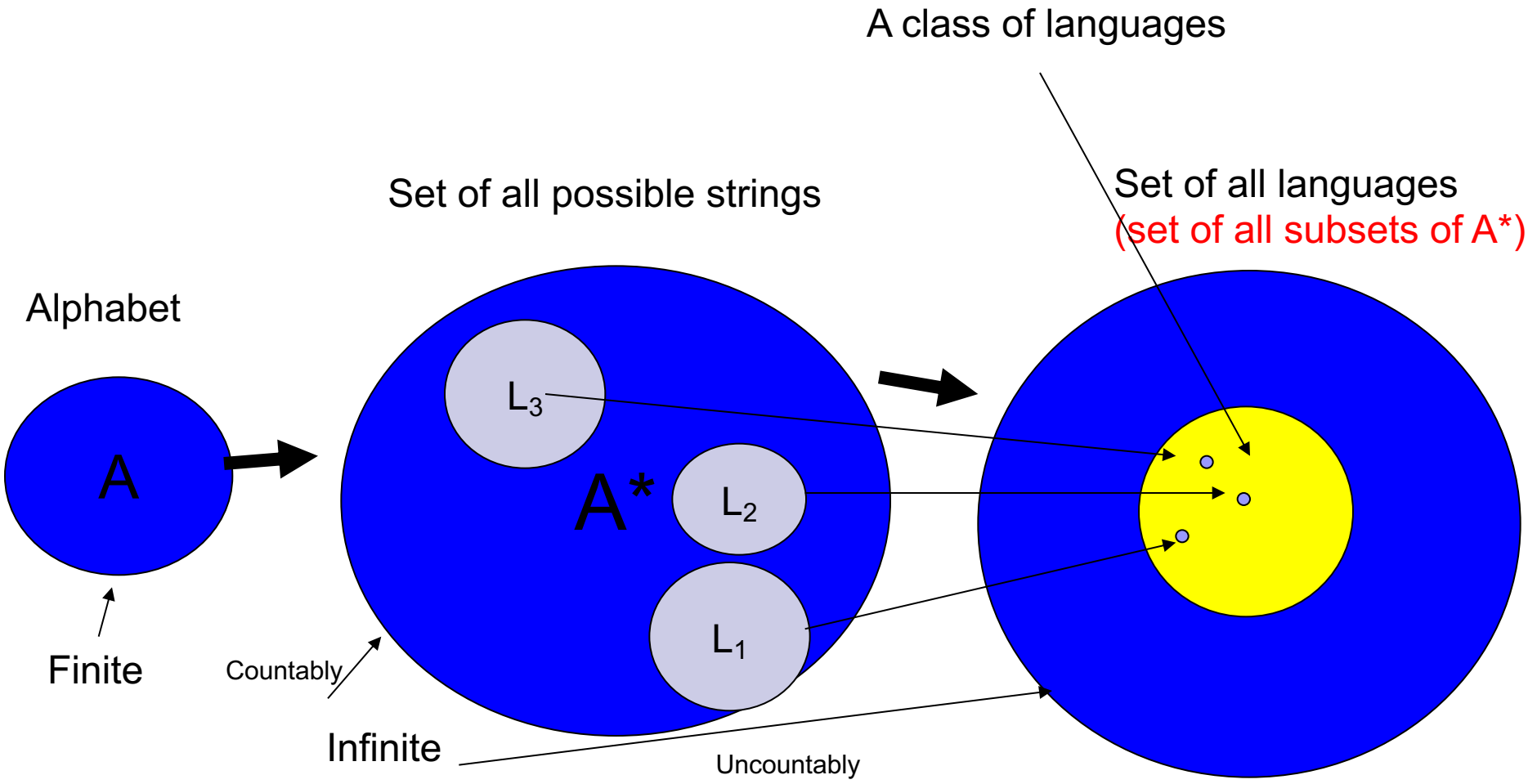
Describing (Formal) Languages

- How do we describe languages?
  - $L = \{a^n b^n : n \geq 0\}$ is fine but not that terribly useful
- We need finite descriptions (of usually infinite sets)
- We need to be able to use these descriptions in "mechanizable" procedures.
  - e.g., to check if a given string is in the language or not

Recognition Problem

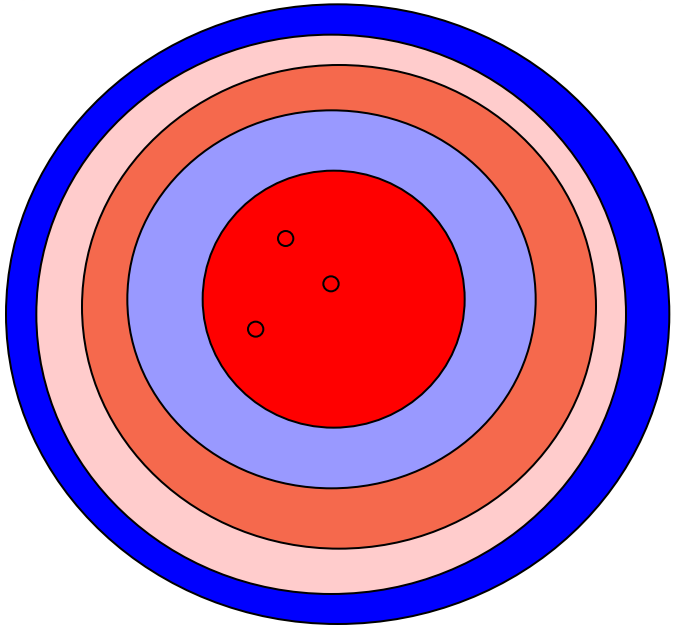
- Given a language L and a string w
  - Is w in L?
  - The problem is that interesting languages are infinite!
  - We need finite descriptions of (infinite) languages.

Classes of Languages



Classes of (Formal) Languages

- Chomksy Hierarchy
  - Regular Languages (simplest)
  - Context Free Languages
  - Context Sensitive Languages
  - Recursively Enumerable Languages



Regular Languages

- Regular languages are those that can be recognized by a finite state recognizer.

Regular Languages

- Regular languages are those that can be recognized by a finite state recognizer.

- What is a finite state recognizer?

Finite State Recognizers

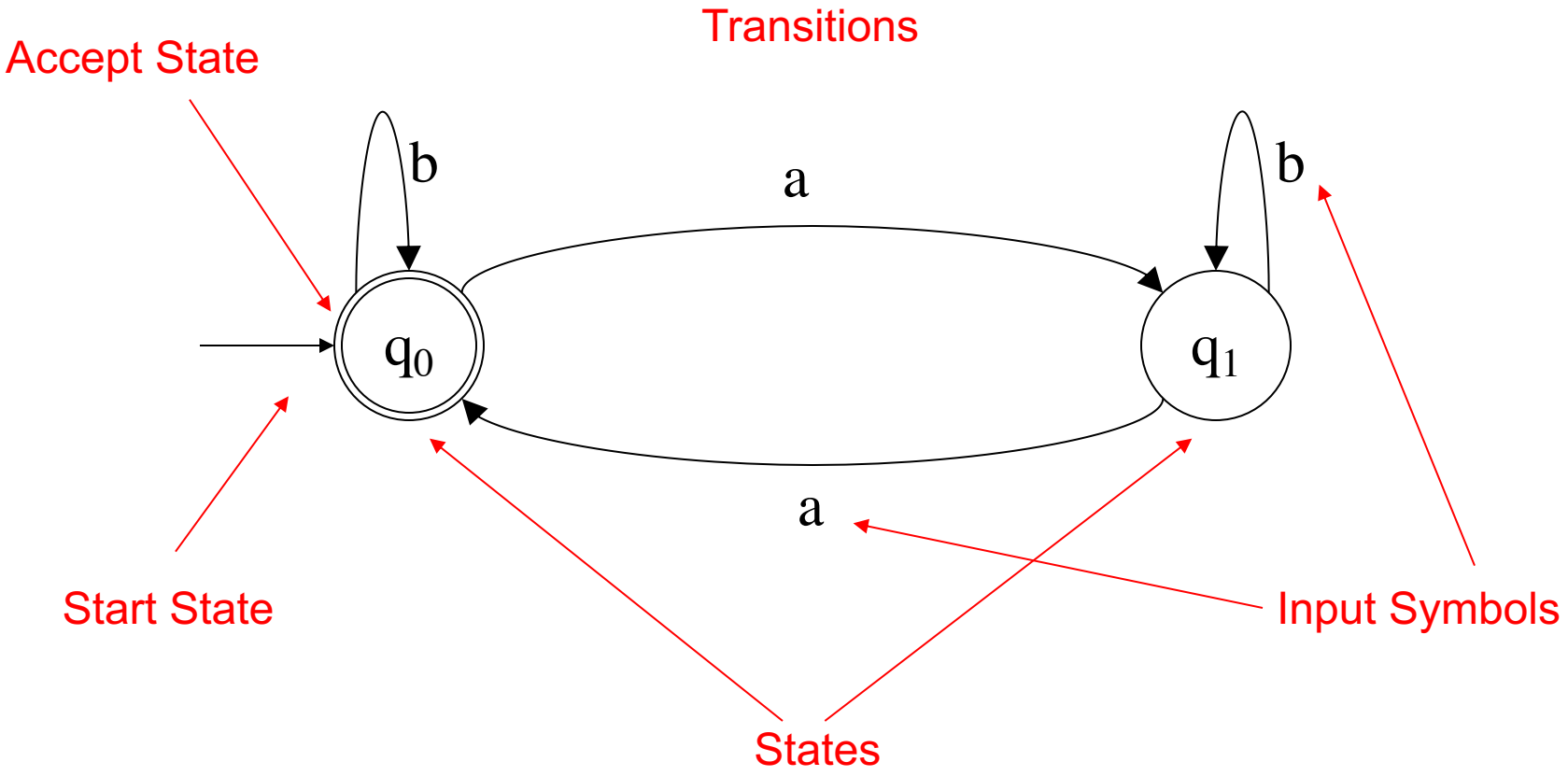
- Consider the mechanization of “recognizing” strings with even number of a’s.
  - Input is a string consisting of a’s and b’s
    - abaabbaaba
  - Count a’s but modulo 2, i.e., when count reaches 2 reset to 0. Ignore the b’s – we are not interested in b’s.
    - $_0a_1b_1a_0a_1b_1b_1a_0a_1b_1a_0$
  - Since we reset to 0 after seeing two a’s, any string that ends with a count 0 matches our condition.

Finite State Recognizers

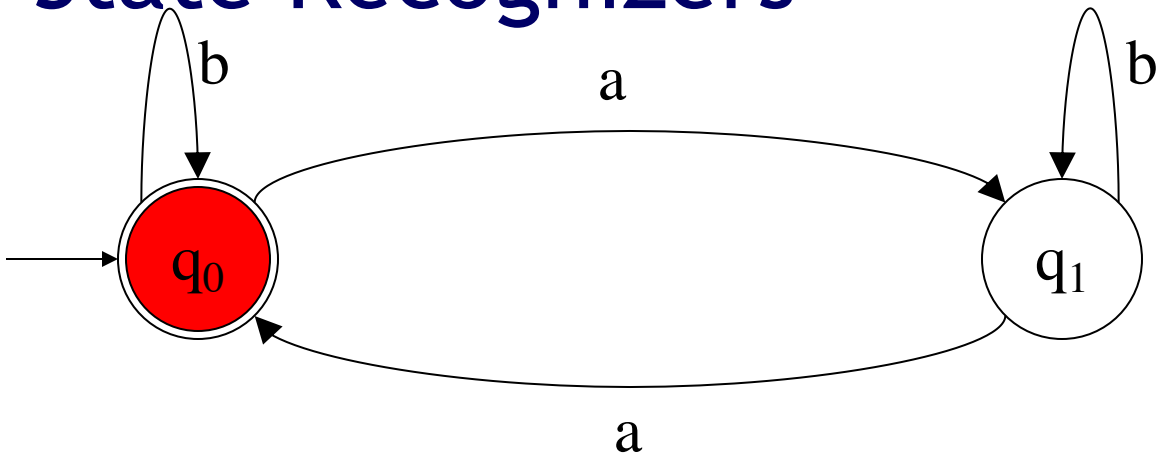
- $_0a_1b_1a_0a_1b_1b_1a_0a_1b_1a_0$
- The symbols shown in between input symbols encode/remember some "interesting" property of the string seen so far.
  - e.g., a 0 shows that we have seen an even number of a's, while a 1, shows an odd number of a's, so far.
- Let's call this "what we remember from past" as the state.
- A "finite state recognizer" can only remember a finite number of distinct pieces of information from the past.

Finite State Recognizers

- We picture FSRs with state graphs.



Finite State Recognizers

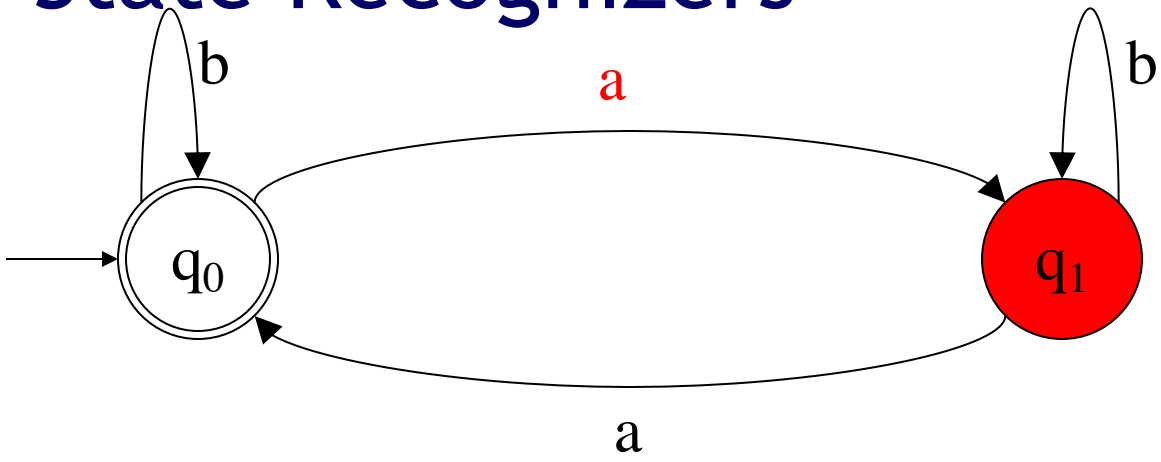


Is abab in the language?

a b a b

^

Finite State Recognizers

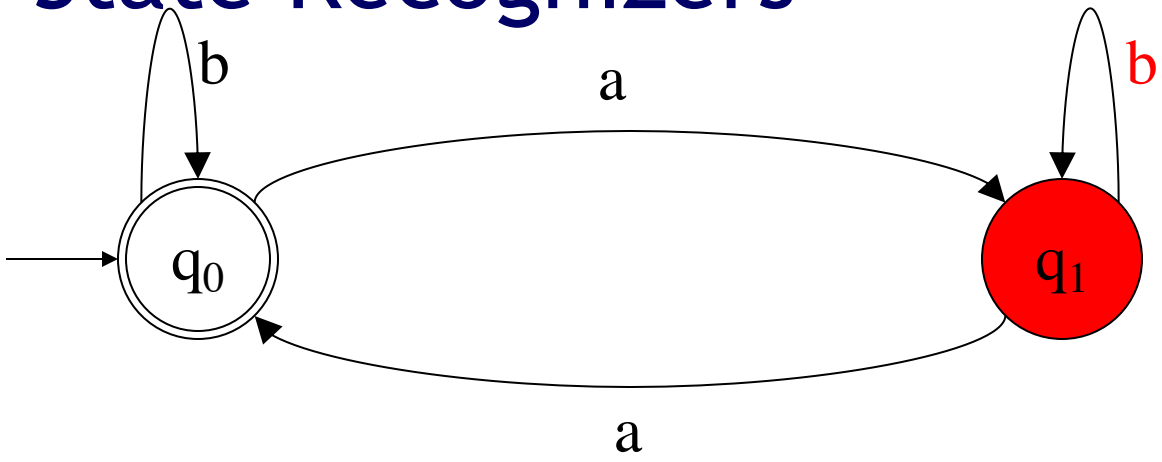


Is abab in the language?

a b a b

^

Finite State Recognizers

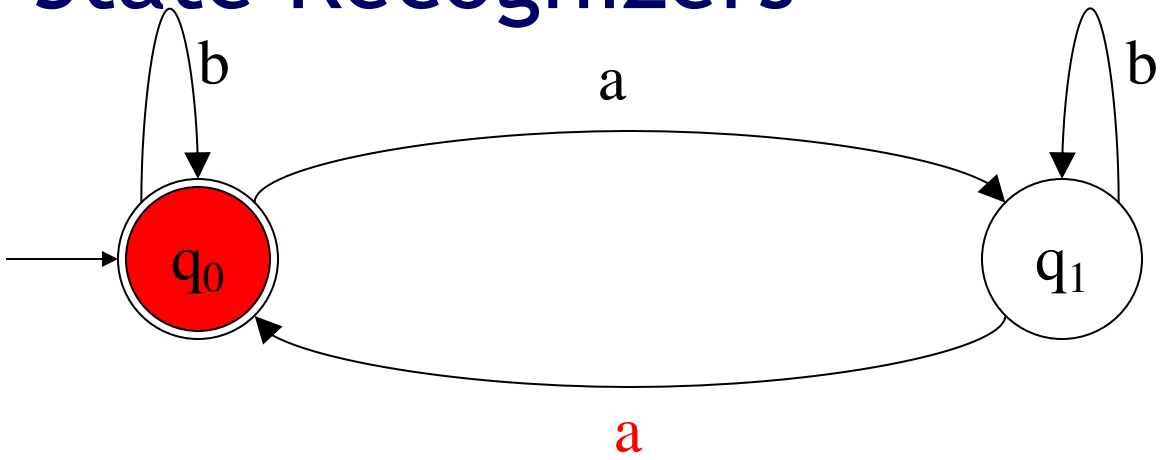


Is abab in the language?

a b a b

^

Finite State Recognizers

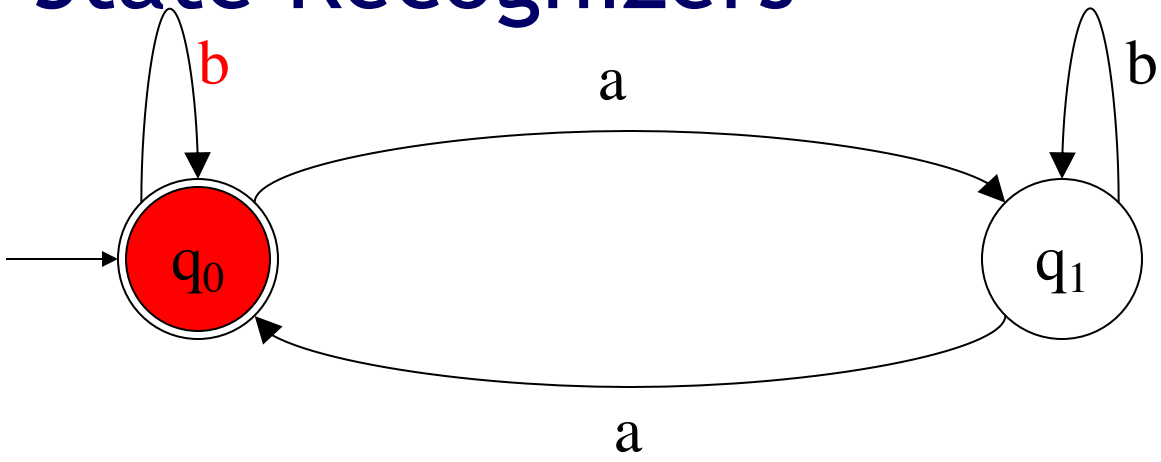


Is abab in the language?

a b a b

^

Finite State Recognizers

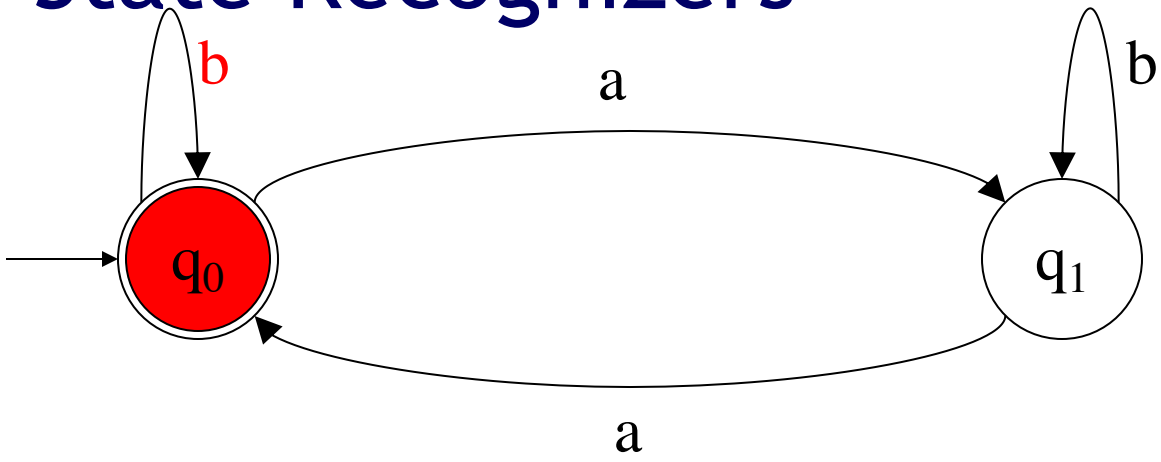


Is abab in the language? YES

a b a b

^

Finite State Recognizers



Is abab in the language? YES

a b a b

^

The state q_0 remembers the fact that we have seen an even number of a's
The state q_1 remembers the fact that we have seen an odd number of a's

Regular Languages

- A language whose strings are accepted by some finite state recognizer is a regular language.

Regular Languages

- A language whose strings are accepted by some finite state recognizer is a regular language.

- However, FSRs can get rather wieldy; we need higher level notations for describing regular languages.

Regular Expressions

- A regular expression is compact formula or metalanguage that describes a regular language.
- For every regular language
  - There are one or more regular expressions
  - There are one or more finite state recognizers
    - But only one minimal finite state recognizer
- Regular expressions can be compiled into finite state recognizers
- Finite state recognizers can be converted to regular expressions.

Constructing Regular Expressions

Expression	Language
Φ	$\{\ \}$
ε	$\{\varepsilon\}$
$\mathbf{a}, \forall a \in A$	$\{a\}$
$R_1 + R_2$	$L_1 \cup L_2$
$R_1\ R_2$	$\{w_1w_2: w_1 \in L_1 \text{ and } w_2 \in L_2\}$
(R)	L

Constructing Regular Expressions

Expression	Language
Φ	$\{\ \}$
ε	$\{\varepsilon\}$
$\mathbf{a}, \forall a \in A$	$\{a\}$
$R_1 + R_2$	$L_1 \cup L_2$
$R_1\ R_2$	$\{w_1w_2: w_1 \in L_1 \text{ and } w_2 \in L_2\}$
(R)	L

a b (a + c) (d + e)

R_1= a b **R_2 = (a + c) (d + e)**

R_{11} =a R_{12} = b **R_{21} = (a + c)** **R_{22}= (d + e)**

Constructing Regular Expressions

Expression	Language
Φ	$\{\ \}$
ε	$\{\varepsilon\}$
$\mathbf{a}, \forall a \in A$	$\{a\}$
$R_1 + R_2$	$L_1 \cup L_2$
$R_1\ R_2$	$\{w_1w_2: w_1 \in L_1 \text{ and } w_2 \in L_2\}$
(R)	L

a b (a + c) (d + e) ⟶ { abad, abae, abcd, abae}

Regular expression represents Regular set/language

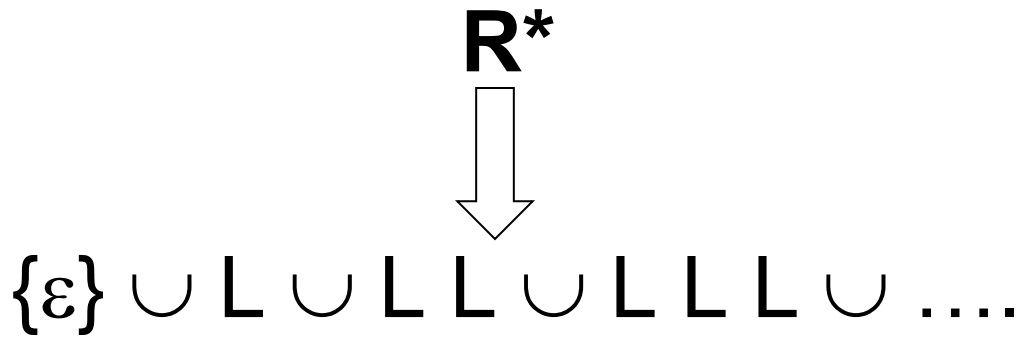
Constructing Regular Expressions

Expression	Language
Φ	$\{\ \}$
ε	$\{\varepsilon\}$
$\mathbf{a}, \forall a \in A$	$\{a\}$
$R_1 + R_2$	$L_1 \cup L_2$
$R_1\ R_2$	$\{w_1w_2: w_1 \in L_1 \text{ and } w_2 \in L_2\}$
(R)	L

- What is the problem/limitation here?

Constructing Regular Expressions

- This much is not that interesting; allows us to describe only finite languages.
- The Kleene Star operator describes infinite sets.



Kleene Star Operator

- **a*** $\Rightarrow \{\varepsilon, a, aa, aaa, aaaa, \ldots..\}$

Kleene Star Operator

- **a*** $\Rightarrow \{\varepsilon, a, aa, aaa, aaaa, \ldots..\}$
- **(ab)*** $\Rightarrow \{\varepsilon, ab, abab, ababab, \ldots\}$

Kleene Star Operator

- **a*** $\Rightarrow$ {ε, a, aa, aaa, aaaa, …..}
- **(ab)*** $\Rightarrow$ {ε, ab, abab, ababab, … }
- **ab*a** $\Rightarrow$ {aa, aba, abba, abbba,…}

Kleene Star Operator

- **a*** $\Rightarrow$ {ε, a, aa, aaa, aaaa, …..}
- **(ab)*** $\Rightarrow$ {ε, ab, abab, ababab, … }
- **ab*a** $\Rightarrow$ {aa, aba, abba, abbba,…}
- **(a+b)*(c+d)*** $\Rightarrow$ {ε, a, b, c, d, ac, ad, bc, bd, aac, abc,bac, bbc, …..}

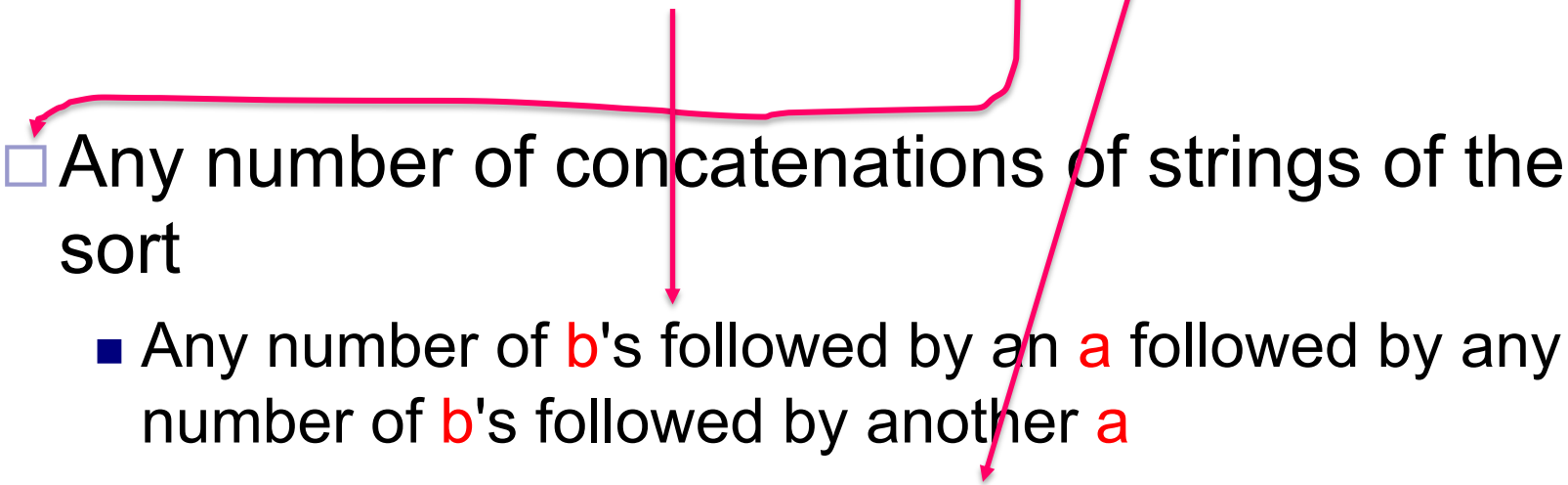
Notational Limitations

- Note that while regular languages are sets there are no operators for
  - Intersection
  - Complementations
- Although
  - Intersection or set difference of two regular languages is a regular language
  - The complement of a regular language is a regular language
- More expressive regular expression formalizations provide such operators but in theory they are not needed.

Regular Expressions

- Regular expression for the set of strings with an even number of a's.

$$(b^* a\ b^* a)^*\ b^*$$



  - Any number of concatenations of strings of the sort
    - Any number of b's followed by an a followed by any number of b's followed by another a
  - Ending with any number of b's

Regular Expressions

- Regular expression for set of strings with an even number of a's.

$$(b^* a b^* a)^* b^*$$

- b b a b a b b a a b a a a b a b b b

  b* a b* a b* a b* a b* a b* a b* a b* a **b***

Any number of strings matching b*ab*a concatenated

Ending with any number of b's

Regular Languages

- Regular languages are described by regular expressions.
- Regular languages are recognized by finite state recognizers.
- Regular expressions define finite state recognizers.

More Examples of Regular Expressions

- All strings ending in a
  - (a+b)*a
- All strings in which the first and the last symbols are the same
  - (a (a+b)* a) + (b (a+b)* b) + a + b
- All strings in which the third symbol from the end is a b
  - (a+b)* b (a+b) (a+b)

Let's see if we can work in reverse

- 0*10*
  - All strings that contain a single 1
- (0+1)*1(0+1)*
  - All strings that contain at least one 1
- 0(0+1)*0 +1(0+1)*1+0+1
  - All strings that start and end with the same symbol.

More Examples

- Strings of the sort $a^n b^m$ with $n + m$ even.
  - (aa)*(bb)*+(aa)*a(bb)*b
- Strings that have at least one 00
  - (0+1)*00(0+1)*
- Strings that have no 00
  - (1+01)* (ε + 0) + (ε + 0) (1 + 10)*;
- Strings that have exactly one 00.
  - (1+01)* 00 (1+10)*

More examples

- The set of strings that contain aba but not abaa in it.
- The set of strings that contain at least one 0 and at most one 1.
  - 00* + 00*10*+100*
- The set of strings that does not contain the substring 100.
- The set of strings where the third symbol from the end is a 1.
  - (0+1)*1(0+1)(0+1)
- The set of strings over {0,1} when intepreted as a binary number, represents a number divisible by 3.
  - 11, 0, 110, 1100 are OK
  - 111, 1000 are not OK
- The set of strings where every pair of consecutive 0's come before any pair of consecutive 1's
  - Multiple cases to consider
  - All 0's
  - Mix of 0s or 1s but no 11 anywhere
  - Has 11 somewhere but no 00 after any 11

Limitations of regular expressions

- Despite all the interesting patterns we could describe, regular expressions area actually quite limited.
- The computational model of finite state recognizers is the most restricted model:
  - There is only a fixed amount of memory $\Longrightarrow$
  - You can only make a fixed finite number of dictinctions as you look at symbols one by one.
  - Input strings can be arbitrarily long, however!

Classes of (Formal) Languages

- Regular Languages (simplest) $\subset$
- Context Free Languages $\subset$
  - Context Free Grammars
- Decidable Languages $\subset$
  - Turing Machines that always halt on every input
- Recursively Enumerable Languages $\subset$
  - Turing Machines that always halt on strings in the language but may or may not halt on strings not in the language
- All languages
  - Some of these languages can be defined but have no computational models to decide or enumerate them.

Limitations of regular expressions

- Regular expressions can not describe languages containing
  - Strings which contain equal numbers of 0's and 1's
  - Strings whole lengths are perfect squares.
  - Strings whose lengths are prime numbers
  - Strings which are palindromes
  - Strings which are valid English sentences.
  - Strings representing Python programs which never get into an infinite loop.

Regular Expressions in Python

- Richer syntactic sugar
- Facility to remember what you matched and then refer to it
  - Not finite state!
- See Python Regex Library Documentation

Regular Expressions in Python

Method/Attribute	Purpose
`match()`	Determine if the RE matches at the beginning of the string.
`search()`	Scan through a string, looking for any location where this RE matches.
`findall()`	Find all substrings where the RE matches, and returns them as a list.
`finditer()`	Find all substrings where the RE matches, and returns them as an iterator.

Python Regex Examples

- (b* a b* a)* b*
  - All strings with an even number a's
- Python
  - `m = re.match('(b*ab*a)*b*','bbbababbbbbaaba')`
  - `<re.Match object; span=(0, 13), match='bbbababbbbbaab'>`
  - `m = re.match('(b*ab*a)*b*$','bbbababbbbbaaba')`
  - `m is a none object (why)`

Python Regex Examples

- (1+01)*00(1+10)*
  - All strings with exactly one 00
- Python
  - `m = re.match('(1|01)*00(1|10)*$', '00');`
  - `<re.Match object; span=(0, 2), match='00'>`
  - `m = re.match('(1|01)*00(1|10)*$', '000')`
  - `Returns none`
  - `m = re.match('(1|01)*00(1|10)*$', '01010100101010111')`
  - `<re.Match object; span=(0, 18), match='01010100101010111'>`